Assignment8 Lab Report

Mrigaj Shaw

2024CSB041

1 Problem1

Q1: Insert data into a table containing two attributes namely radius & circumference of circles. You may get different values of radius either from keyboard or you may generate different values.

```
DO $$
DECLARE
    r NUMERIC(10, 2);
    circ NUMERIC(10, 2);
BEGIN
    FOR r in 1..20 LOOP
        circ := 2 * pi() * r;

        INSERT INTO circles (radius, circumference)
        VALUES (r, circ);
    END LOOP;
END $$;

SELECT * FROM circles ORDER BY radius;
```

Statement executed successfully.

Output:

radius	**circumference**
1.00	6.28
2.00	12.57
3.00	18.85
4.00	25.13
5.00	31.42
6.00	37.70
7.00	43.98
8.00	50.27
9.00	56.55
10.00	62.83
11.00	69.12
12.00	75.40
13.00	81.68
14.00	87.96
15.00	94.25
16.00	100.53
17.00	106.81
18.00	113.10
19.00	119.38
20.00	125.66

Q2: Update the balance of each customer from a cust_acct table showing withdrawal of Rs.1000/- as service charge provided that the customer balance shows at least Rs.1000/-.

```
SELECT * FROM cust_acct ORDER BY cust_id;

DO $$
DECLARE c RECORD;
BEGIN
    FOR c in SELECT cust_id, balance FROM cust_acct LOOP
        IF c.balance >= 1000 THEN
            UPDATE cust_acct SET balance = balance - 1000
            WHERE cust_id = c.cust_id;
        END IF;
    END LOOP;
END $$;

SELECT * FROM cust_acct ORDER BY cust_id;
```

Output:

cust_id	cust_name	balance
1	Amit	5000.00
2	Bina	1000.00
3	Chirag	750.00
4	Deepa	25000.00
5	Esha	999.99

Statement executed successfully.

Output:

cust_id	cust_name	balance
1	Amit	4000.00
2	Bina	0.00
3	Chirag	750.00
4	Deepa	24000.00
5	Esha	999.99

Q3: Update the salary of each employee from EMP table by 15% using cursor.

```
SELECT * FROM employees ORDER BY emp_id;

DO $$
DECLARE
    cur CURSOR FOR SELECT emp_id, salary from employees;
    rec RECORD;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur into rec;
        EXIT WHEN NOT FOUND;
        UPDATE employees SET salary = salary * 1.15 WHERE emp_id = rec.
           emp_id;
    END LOOP;
    CLOSE cur;
END $$;

SELECT * FROM employees ORDER BY emp_id;
```

Output:

emp_id	emp_name	salary
101	Rahul	50000.00
102	Sneha	60000.00
103	Kiran	45000.00
104	Anita	72000.00
105	Vikram	NULL

Statement executed successfully.

Output:

emp_id	emp_name	salary
101	Rahul	57500.00
102	Sneha	69000.00
103	Kiran	51750.00
104	Anita	82800.00
105	Vikram	NULL

Q4: Update the balance in the ITEM_MSTR table each time a transaction takes place in the ITEM_TR table. If this item_id is already present in the ITEM_MSTR table an update is performed to decrease the balance by the quantity specified in the ITEM_TR table. If the item_id is not present in the ITEM_MSTR table, the tuple is to be inserted.

```
SELECT * FROM item_mstr ORDER BY item_id;

CREATE OR REPLACE FUNCTION trg_update_item() RETURNS TRIGGER AS $$
BEGIN
    UPDATE item_mstr SET balance = balance - NEW.quantity WHERE item_id
        = NEW.item_id;

    IF NOT FOUND THEN
        INSERT INTO item_mstr (item_id, balance)
        VALUES (NEW.item_id, NEW.quantity);
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER after_item_tr
AFTER INSERT on item_tr
FOR EACH ROW EXECUTE FUNCTION trg_update_item();

INSERT INTO item_tr (item_id, quantity) VALUES (1, 10);
INSERT INTO item_tr (item_id, quantity) VALUES (9, 7);

SELECT * FROM item_mstr ORDER BY item_id;
```

Output:

item_id	balance
1	100
2	50
3	75

Statement executed successfully.

Statement executed successfully.

Statement executed successfully. Rows affected: 1

Statement executed successfully. Rows affected: 1

Output:

item_id	balance
1	90
2	50
3	75
9	7

Q5: Write a PROCEDURE for raising salary of some employee by some amount. The PROCEDURE to be written may carry two parameters emp_id and amt to be raised. Include two exceptions which will be raised when either emp_id is not present or salary is NULL.

```
CREATE OR REPLACE PROCEDURE raise_salary(p_emp_id INT, amt NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    sal employees.salary%type;
BEGIN
    SELECT salary INTO sal FROM employees WHERE emp_id = p_emp_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Employee % not found', p_emp_id USING ERRCODE
           = 'P0002';
    ELSEIF sal IS NULL THEN
        RAISE EXCEPTION 'Salary is null for employee %', p_emp_id USING
            ERRCODE = 'P0003';
    END IF;

    UPDATE employees SET salary = salary + amt WHERE emp_id = p_emp_id;
    RAISE NOTICE 'New Salary: %', sal + amt;
EXCEPTION
    WHEN SQLSTATE 'P0002' THEN RAISE NOTICE 'emp_id is not present';
    WHEN SQLSTATE 'P0003' THEN RAISE NOTICE 'Salary of employee is null
       ';
END $$;

CALL raise_salary(101, 5000);
CALL raise_salary(999, 5000);
CALL raise_salary(105, 5000);

SELECT * FROM employees ORDER BY emp_id;
```

Statement executed successfully.

Statement executed successfully.

Notice: New Salary: 55000.00

Statement executed successfully.

Notice: emp_id is not present

Statement executed successfully.

Notice: Salary of employee is null

Output:

emp_id	emp_name	salary
101	Rahul	55000.00
102	Sneha	60000.00
103	Kiran	45000.00
104	Anita	72000.00
105	Vikram	NULL